

Staging buffer

- [Introduction](#)
- [Transfer queue](#)
- [Abstracting buffer creation](#)
- [Using a staging buffer](#)
- [Conclusion](#)

Introduction

The vertex buffer we have right now works correctly, but the memory type that allows us to access it from the CPU may not be the most optimal memory type for the graphics card itself to read from. The most optimal memory has the `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` flag and is usually not accessible by the CPU on dedicated graphics cards. In this chapter we're going to create two vertex buffers. One *staging buffer* in CPU accessible memory to upload the data from the vertex array to, and the final vertex buffer in device local memory. We'll then use a buffer copy command to move the data from the staging buffer to the actual vertex buffer.

Transfer queue

The buffer copy command requires a queue family that supports transfer operations, which is indicated using `VK_QUEUE_TRANSFER_BIT`. The good news is that any queue family with `VK_QUEUE_GRAPHICS_BIT` or `VK_QUEUE_COMPUTE_BIT` capabilities already implicitly support `VK_QUEUE_TRANSFER_BIT` operations. The implementation is not required to explicitly list it in `queueFlags` in those cases.

If you like a challenge, then you can still try to use a different queue family specifically for transfer operations. It will require you to make the following modifications to your program:

- Modify `QueueFamilyIndices` and `findQueueFamilies` to explicitly look for a queue family with the `VK_QUEUE_TRANSFER_BIT` bit, but not the `VK_QUEUE_GRAPHICS_BIT`.
- Modify `createLogicalDevice` to request a handle to the transfer queue
- Create a second command pool for command buffers that are submitted on the transfer queue family
- Change the `sharingMode` of resources to be `VK_SHARING_MODE_CONCURRENT` and specify both the graphics and transfer queue families
- Submit any transfer commands like `vkCmdCopyBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdCopyBuffer.html>) (which we'll be using in this chapter) to the transfer queue instead of the graphics queue

It's a bit of work, but it'll teach you a lot about how resources are shared between queue families.

Abstracting buffer creation

Because we're going to create multiple buffers in this chapter, it's a good idea to move buffer creation to a helper function. Create a new function `createBuffer` and move the code in `createVertexBuffer` (except mapping) to it.

```

void createBuffer(VkDeviceSize size, VkBufferUsageFlags usage, VkMemoryPropertyFlags properties,
VkBuffer& buffer, VkDeviceMemory& bufferMemory) {
    VkBufferCreateInfo bufferInfo{};
    bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
    bufferInfo.size = size;
    bufferInfo.usage = usage;
    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;

    if (vkCreateBuffer(device, &bufferInfo, nullptr, &buffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to create buffer!");
    }

    VkMemoryRequirements memRequirements;
    vkGetBufferMemoryRequirements(device, buffer, &memRequirements);

    VkMemoryAllocateInfo allocInfo{};
    allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
    allocInfo.allocationSize = memRequirements.size;
    allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, properties);

    if (vkAllocateMemory(device, &allocInfo, nullptr, &bufferMemory) != VK_SUCCESS) {
        throw std::runtime_error("failed to allocate buffer memory!");
    }

    vkBindBufferMemory(device, buffer, bufferMemory, 0);
}

```

Make sure to add parameters for the buffer size, memory properties and usage so that we can use this function to create many different types of buffers. The last two parameters are output variables to write the handles to.

You can now remove the buffer creation and memory allocation code from `createVertexBuffer` and just call `createBuffer` instead:

```

void createVertexBuffer() {
    VkDeviceSize bufferSize = sizeof(vertices[0]) * vertices.size();
    createBuffer(bufferSize, VK_BUFFER_USAGE_VERTEX_BUFFER_BIT, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, vertexBuffer, vertexBufferMemory);

    void* data;
    vkMapMemory(device, vertexBufferMemory, 0, bufferSize, 0, &data);
    memcpy(data, vertices.data(), (size_t) bufferSize);
    vkUnmapMemory(device, vertexBufferMemory);
}

```

Run your program to make sure that the vertex buffer still works properly.

Using a staging buffer

We're now going to change `createVertexBuffer` to only use a host visible buffer as temporary buffer and use a device local one as actual vertex buffer.

```
void createVertexBuffer() {
    VkDeviceSize bufferSize = sizeof(vertices[0]) * vertices.size();

    VkBuffer stagingBuffer;
    VkDeviceMemory stagingBufferMemory;
    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer, stagingBufferMemory);

    void* data;
    vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0, &data);
    memcpy(data, vertices.data(), (size_t) bufferSize);
    vkUnmapMemory(device, stagingBufferMemory);

    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, vertexBuffer, vertexBufferMemory);
}
```

We're now using a new `stagingBuffer` with `stagingBufferMemory` for mapping and copying the vertex data. In this chapter we're going to use two new buffer usage flags:

- `VK_BUFFER_USAGE_TRANSFER_SRC_BIT`: Buffer can be used as source in a memory transfer operation.
- `VK_BUFFER_USAGE_TRANSFER_DST_BIT`: Buffer can be used as destination in a memory transfer operation.

The `vertexBuffer` is now allocated from a memory type that is device local, which generally means that we're not able to use `vkMapMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkMapMemory.html>). However, we can copy data from the `stagingBuffer` to the `vertexBuffer`. We have to indicate that we intend to do that by specifying the transfer source flag for the `stagingBuffer` and the transfer destination flag for the `vertexBuffer`, along with the vertex buffer usage flag.

We're now going to write a function to copy the contents from one buffer to another, called `copyBuffer`.

```
void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize size) {

}
```

Memory transfer operations are executed using command buffers, just like drawing commands. Therefore we must first allocate a temporary command buffer. You may wish to create a separate command pool for these kinds of short-lived buffers, because the implementation may be able to apply memory allocation optimizations. You should use the `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT` flag during command pool generation in that case.

```
void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize size) {  
    VkCommandBufferAllocateInfo allocInfo{};  
    allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;  
    allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;  
    allocInfo.commandPool = commandPool;  
    allocInfo.commandBufferCount = 1;  
  
    VkCommandBuffer commandBuffer;  
    vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer);  
}
```

And immediately start recording the command buffer:

```
VkCommandBufferBeginInfo beginInfo{};  
beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;  
beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;  
  
vkBeginCommandBuffer(commandBuffer, &beginInfo);
```

We're only going to use the command buffer once and wait with returning from the function until the copy operation has finished executing. It's good practice to tell the driver about our intent using

```
VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT .
```

```
VkBufferCopy copyRegion{};  
copyRegion.srcOffset = 0; // Optional  
copyRegion.dstOffset = 0; // Optional  
copyRegion.size = size;  
vkCmdCopyBuffer(commandBuffer, srcBuffer, dstBuffer, 1, &copyRegion);
```

Contents of buffers are transferred using the `vkCmdCopyBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdCopyBuffer.html>) command. It takes the source and destination buffers as arguments, and an array of regions to copy. The regions are defined in `VkBufferCopy` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkBufferCopy.html>) structs and consist of a source buffer offset, destination buffer offset and size. It is not possible to specify `VK_WHOLE_SIZE` here, unlike the `vkMapMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkMapMemory.html>) command.

```
vkEndCommandBuffer(commandBuffer);
```

This command buffer only contains the copy command, so we can stop recording right after that. Now execute the command buffer to complete the transfer:

```
VkSubmitInfo submitInfo{};
submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
submitInfo.commandBufferCount = 1;
submitInfo.pCommandBuffers = &commandBuffer;

vkQueueSubmit(graphicsQueue, 1, &submitInfo, VK_NULL_HANDLE);
vkQueueWaitIdle(graphicsQueue);
```

Unlike the draw commands, there are no events we need to wait on this time. We just want to execute the transfer on the buffers immediately. There are again two possible ways to wait on this transfer to complete. We could use a fence and wait with `vkWaitForFences` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkWaitForFences.html>), or simply wait for the transfer queue to become idle with `vkQueueWaitIdle` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkQueueWaitIdle.html>). A fence would allow you to schedule multiple transfers simultaneously and wait for all of them complete, instead of executing one at a time. That may give the driver more opportunities to optimize.

```
vkFreeCommandBuffers(device, commandPool, 1, &commandBuffer);
```

Don't forget to clean up the command buffer used for the transfer operation.

We can now call `copyBuffer` from the `createVertexBuffer` function to move the vertex data to the device local buffer:

```
createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, vertexBuffer, vertexBufferMemory);

copyBuffer(stagingBuffer, vertexBuffer, bufferSize);
```

After copying the data from the staging buffer to the device buffer, we should clean it up:

```
...

copyBuffer(stagingBuffer, vertexBuffer, bufferSize);

vkDestroyBuffer(device, stagingBuffer, nullptr);
vkFreeMemory(device, stagingBufferMemory, nullptr);
}
```

Run your program to verify that you're seeing the familiar triangle again. The improvement may not be visible right now, but its vertex data is now being loaded from high performance memory. This will matter when we're going to start rendering more complex geometry.

Conclusion

It should be noted that in a real world application, you're not supposed to actually call `vkAllocateMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkAllocateMemory.html>) for every individual buffer. The maximum number of simultaneous memory allocations is limited by the

`maxMemoryAllocationCount` physical device limit, which may be as low as `4096` even on high end hardware like an NVIDIA GTX 1080. The right way to allocate memory for a large number of objects at the same time is to create a custom allocator that splits up a single allocation among many different objects by using the `offset` parameters that we've seen in many functions.

You can either implement such an allocator yourself, or use the [VulkanMemoryAllocator](https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator) (<https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator>) library provided by the GPUOpen initiative. However, for this tutorial it's okay to use a separate allocation for every resource, because we won't come close to hitting any of these limits for now.

[C++ code \(../code/20_staging_buffer.cpp\)](#) / [Vertex shader \(../code/18_shader_vertexbuffer.vert\)](#) / [Fragment shader \(../code/18_shader_vertexbuffer.frag\)](#)

[Previous \(/en/Vertex_buffers/Vertex_buffer_creation\)](/en/Vertex_buffers/Vertex_buffer_creation)[Next \(/en/Vertex_buffers/Index_buffer\)](/en/Vertex_buffers/Index_buffer)

ALSO ON VULKAN TUTORIAL

Vulkan Tutorial

10 years ago · 48 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Swap chain recreation

10 years ago · 219 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Descriptor layout and buffer

10 years ago · 147 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Vulkan T

10 years ag

A tutorial t everything 3D graphic